GEOTAG STUDIO

Engineering Improvement & Feature Specification Prompt

Project	GeoTag Studio — Open-Source Browser-Based GPS Metadata Editor
Repository	https://github.com/balramsharma1725/GeoTag-Studio
Hosted At	https://balramsharma1725.github.io/GeoTag-Studio/
Tech Stack	React • Vite • TypeScript • Tailwind CSS • Zustand • React-Leaflet • piexifjs • JSZip • Framer Motion
License	MIT • Fully client-side • Privacy-first (no server uploads)

1. Role & Persona

You are a Senior Full-Stack Engineer with deep expertise in React 18+, Vite, TypeScript, and browser-native APIs. You specialize in privacy-first, offline-capable web applications that require zero server infrastructure. You write clean, modular, well-documented code that follows WCAG 2.1 accessibility standards, performs reliably across devices, and handles edge cases gracefully.

Your responses are structured as professional engineering deliverables: each change includes a rationale, full code snippets or diffs, edge-case analysis, and concrete testing guidance. You never compromise on the project's core ethos: all data processing stays client-side and nothing is uploaded to external servers.

2. Project Context & Current Capabilities

2.1 What GeoTag Studio Does

GeoTag Studio is a browser-based tool that lets users read, edit, and write GPS metadata (geotags) and EXIF data to images — entirely locally. Users drag-and-drop JPG, PNG, or WEBP images, select a location on an interactive OpenStreetMap-powered map (or type coordinates manually), and export the modified files individually or as a ZIP archive.

2.2 Existing Features

- Drag-and-drop image upload (JPG, PNG, WEBP)
- Interactive map (React-Leaflet + OpenStreetMap) for GPS selection
- Batch GPS editing across multiple images
- EXIF read/write via piexifjs and exifr
- ZIP export via JSZip
- Undo/reset and before-after preview
- Dark mode UI
- Animations via Framer Motion

3. New Feature: Per-Image Alt Text Editor

> ✨ **Priority Feature Request**
> Users currently cannot assign unique alt text (or IPTC image descriptions) to individual images in a batch. This feature adds a dedicated per-image alt text input panel, allowing each image to carry its own accessibility description and SEO metadata — all processed locally without any server calls.

3.1 Feature Requirements

Core Behavior

- Each image in the workspace must have its own independent alt text field.
- The field must pre-populate from any existing EXIF ImageDescription or IPTC Caption/Abstract already present in the file.
- Changes must be persisted to the Zustand store immediately (no save button required for the field itself).
- On export (single file or ZIP), the alt text value must be written to both EXIF ImageDescription and IPTC Caption-Abstract using piexifjs.
- Bulk-fill option: a “Apply to All” button propagates the currently focused image’s alt text to all other images that have an empty alt text field (does NOT overwrite existing values).
- Character counter displayed alongside the field (recommended: warn at 100 chars, hard limit at 250).

UI Placement

- On desktop (≥ 768 px): render the alt text input below the existing filename/coordinates panel for the selected image in the right sidebar.
- On mobile (< 768 px): render it in the image detail drawer that expands on image tap.
- Use a <textarea> (not <input>) to accommodate multi-line descriptions; auto-resize on content change.

Accessibility Requirements

- aria-label="Alt text for [filename]" on each textarea.
- Role announcements for character-count warnings via aria-live="polite".
- Keyboard focus management: Tab moves from coordinates input → alt text → export button.

3.2 State Management (Zustand)

Extend the existing image store slice. The property name must match what piexifjs writes to EXIF:

```
// types/image.ts
interface ImageEntry {
  id: string;
  file: File;
  lat: number | null;
  lng: number | null;
  altText: string;          // <-- add this
  // ...existing fields
}
```

```
// store/imageStore.ts  —  new action
setAltText: (id: string, value: string) => void;
applyAltTextToEmpty: (sourceId: string) => void;
```

3.3 Component: AltTextPanel

Create a new component at src/components/AltTextPanel.tsx. It receives the image id as a prop and reads/writes via the Zustand store:

```
// src/components/AltTextPanel.tsx
import { useImageStore } from '../store/imageStore';

interface Props { imageId: string; filename: string; }

export const AltTextPanel = ({ imageId, filename }: Props) => {
  const { images, setAltText, applyAltTextToEmpty } = useImageStore();
  const current = images.find(img => img.id === imageId);
  const value = current?.altText ?? '';
  const MAX = 250; const WARN = 100;

  return (
    <div className="flex flex-col gap-2 mt-3">
      <label className="text-xs font-semibold text-slate-400 uppercase tracking-
wide">
        Alt Text / Description
```

```
      </label>
      <textarea
        aria-label={`Alt text for ${filename}`}
        maxLength={MAX}
        rows={3}
        value={value}
        onChange={e => setAltText(imageId, e.target.value)}
        className="w-full resize-none rounded-md border border-slate-600 bg-slate-
800
                   px-3 py-2 text-sm text-slate-200 focus:ring-2 focus:ring-blue-
500"
        placeholder="Describe the image for accessibility and SEO..."
      />
      <div className="flex justify-between items-center text-xs">
        <span aria-live="polite"
              className={value.length >= WARN ? 'text-amber-400' : 'text-slate-
500'}>
          {value.length}/{MAX} characters
        </span>
        <button onClick={() => applyAltTextToEmpty(imageId)}
                className="text-blue-400 hover:text-blue-300 transition-colors">
          Apply to empty images
        </button>
      </div>
    </div>
  );
};
```

3.4 EXIF Write Logic (exifUtils.ts)

Extend the existing writeExif utility to also write alt text to ImageDescription (0x010E) and, if IPTC support is available, to the IPTC Caption-Abstract tag:

```
// utils/exifUtils.ts  —  inside your existing writeGpsAndMetadata function

if (altText && altText.trim().length > 0) {
  // EXIF ImageDescription (tag 0x010E) — ASCII, max 255 bytes
  const ascii = altText.normalize('NFC').slice(0, 255);
  exifObj['0th'][piexif.ImageIFD.ImageDescription] = ascii;

  // EXIF UserComment — stores full UTF-8 with charset prefix
  // Format: 8-byte charset code + actual string
  const encoder = new TextEncoder();
  const prefix = new Uint8Array([85,78,73,67,79,68,69,0]); // 'UNICODE\0'
  const encoded = encoder.encode(altText);
  const userComment = new Uint8Array(prefix.length + encoded.length);
  userComment.set(prefix);
```

```
  userComment.set(encoded, prefix.length);
  exifObj['Exif'][piexif.ExifIFD.UserComment] = Array.from(userComment);
}
```

Note on UTF-16LE encoding fix: piexifjs writes string fields as ASCII by default, which silently drops non-ASCII characters. The UserComment field above explicitly prepends the Unicode charset marker, preserving Devanagari, CJK, and other special characters correctly.

3.5 Testing Checklist

1. Load an image with an existing ImageDescription — confirm the field pre-populates.
2. Type a new alt text, export, re-import the file, and verify the value round-trips correctly.
3. Enter a non-ASCII string (e.g., Hindi, Arabic, emoji) — confirm it exports and re-reads without corruption.
4. Exceed 100 characters — confirm amber warning appears. Exceed 250 — confirm input is capped.
5. Click “Apply to empty images” — verify only images with empty alt text fields are updated.
6. Tab through the panel using keyboard only — verify focus order is logical.
7. Test on a mobile viewport (375 px) — confirm textarea is usable and keyboard doesn’t obscure content.
8. Run Vitest unit test: mock a Zustand store, call setAltText, assert state updates correctly.

4. Existing Problem Fixes — Prioritized

In addition to the new Alt Text feature, address the following previously reported issues. For each fix, provide a description, the root cause, a code snippet or diff, and testing steps.

#	Area	What to Implement	Why It Matters
1	**Offline Map Support**	Integrate leaflet-offline or a Service Worker tile cache. Provide fallback to manual lat/lng entry when tiles are unavailable.	Journalists and field photographers often work without connectivity. Offline maps keep the tool usable anywhere.
2	**CSV / GPX Coordinate Import**	Parse CSV (filename, lat, lng, keywords, description) and GPX tracks. Show a preview modal before applying. Match by filename or sequential order.	Batch workflows with pre-surveyed coordinates are common for real estate, journalism, and local SEO.
3	**Expanded Metadata Fields**	Add timestamp editing, IPTC/XMP fields. Fix UTF-16LE encoding bugs for non-ASCII text in keyword and description fields.	Professional photographers and stock agencies require accurate timestamps and IPTC compliance.

#	Area	What to Implement	Why It Matters
4	**Theme Toggle (Light / Dark / System)**	Add a theme switcher using Tailwind dark: variants. Persist selection to localStorage. Update map tiles to match theme.	Light mode is essential for daytime outdoor use and for users with visual sensitivities to dark UIs.
5	**Mobile & PWA Optimization**	Responsive layout with collapsible panels, touch-optimized map controls, virtual keyboard handling. Add vite-plugin-pwa for installable PWA.	Many photographers use mobile devices in the field. A PWA allows offline install and home screen access.
6	**Accessibility (WCAG 2.1 AA)**	Add ARIA attributes, keyboard navigation for map markers, screen reader announcements for preview changes, color contrast fixes. Target 90+ Lighthouse score.	Accessibility is a legal requirement in many jurisdictions and widens the potential user base.
7	**Large-Batch Performance**	Use Web Workers for EXIF processing. Add lazy-loaded image previews. Implement chunked ZIP export with progress bar and cancel button.	Processing 100+ images synchronously blocks the main thread, causing freezes. Workers eliminate this entirely.
8	**Broader Format Support**	Handle HEIC/HEIF (heic2any), TIFF (utif), and RAW formats (CR2, NEF) with graceful degradation for unsupported files.	iPhone users shoot HEIC by default. RAW support is critical for professional photographers.
9	**Error Handling & Testing**	Validate coordinates, detect corrupted files, show user-friendly toast alerts. Add Vitest unit tests for EXIF write, batch processing, and coordinate parsing.	Silent failures erode user trust. Tests prevent regressions as the codebase grows.
10	**Documentation & Help UX**	In-app help modal with tooltips. Update README with screenshots and usage examples. Add CONTRIBUTING.md for open-source contributors.	Good docs reduce support burden and encourage community contributions.

5. Required Output Format

For every improvement and the new Alt Text feature, structure your response as follows:

Response Structure (repeat for each feature)

Overview → New dependencies (npm install ...) → Description & Rationale → Code updates (diff-style or full file) → Edge cases handled → Testing steps

5.1 Code Quality Standards

- TypeScript strict mode ("strict": true in tsconfig.json). No implicit any.
- Functional React components with hooks only. No class components.
- Zustand for all global state. No prop-drilling beyond 2 levels.
- Tailwind utility classes only — no inline styles, no custom CSS unless unavoidable.
- All browser APIs (IndexedDB, Service Workers, Web Workers) must be feature-detected before use.
- No external API calls, no telemetry, no analytics. Everything stays in-browser.
- All async operations must have explicit error boundaries with user-facing fallback messages.

5.2 Constraints

- Deployment target is GitHub Pages (static hosting). No server-side rendering, no backend.
- Bundle size budget: < 500 KB gzipped for initial load. Lazy-import heavy libraries (e.g., heic2any, utif).
- Support: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+. Mobile: iOS Safari 14+, Chrome Android 90+.
- All features must degrade gracefully on unsupported browsers — show a clear warning, never crash silently.

---

6. Deployment Guide (GitHub Pages)

6.1 Build & Deploy Steps

9. Ensure vite.config.ts has base: '/GeoTag-Studio/' set for the subdirectory path.
10. Run npm run build — output goes to dist/.
11. Deploy using gh-pages: npx gh-pages -d dist or configure the GitHub Actions workflow below.
12. Verify the deployed site at https://balramsharma1725.github.io/GeoTag-Studio/

6.2 GitHub Actions Workflow

```
# .github/workflows/deploy.yml
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
```

```
        with: { node-version: '18' }
      - run: npm ci
      - run: npm run build
      - uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: ./dist
```

---

7. Future Expansions (Post v2.0)

Electron Desktop Wrapper: Package GeoTag Studio as a native desktop app using Electron + electron-builder. This enables direct filesystem access (no file size limits), OS-native file dialogs, and menubar integration. Share 100% of the existing React codebase; add only a thin Electron main process.

AI-Assisted Metadata: Integrate a WebGPU-accelerated local model (e.g., Moondream via WebAssembly) to auto-suggest alt text and keywords from image content — entirely in-browser, no API keys required.

Collaborative Presets: Allow users to export/import named coordinate presets and metadata templates as JSON files, enabling team-wide consistency without a backend.

Reverse Geocoding Panel: When a user drops a pin on the map, auto-fill a human-readable address using the Nominatim API (no key required) and offer to include it as an IPTC Location field.

Google Photos / iCloud Import: Explore OAuth-based import from cloud photo libraries to allow direct editing without manual file downloads.

End of Specification — GeoTag Studio v2.0 Engineering Prompt